We've made a case for reinforcement learning, but why *deep* reinforcement learning? RL existed long before the popular rise of deep learning. In fact, some of the earliest methods (which we will look at for learning purposes) involved nothing more than storing experiences in a lookup table (e.g., a Python dictionary) and updating that table on each iteration of the algorithm. The idea was to let the agent play around in the environment and see what happened, and to store its experiences of what happened in some sort of database. After a while, you could look back on this database of knowledge and observe what worked and what didn't. No neural networks or other fancy algorithms.

For very simple environments this actually works fairly well. For example, in Tic-Tac-Toe there are 255,168 valid board positions. The *lookup table* (also called a *memory table*) would have that many entries, which mapped from each state to a specific action (as shown in figure 1.12) and the reward observed (not depicted). During training, the algorithm could learn which move led toward more favorable positions and update that entry in the memory table.

Figure 1.12. An action lookup table for Tic-Tac-Toe with only three entries, where the "player" (an algorithm) plays X. When the player is given a board position, the lookup table dictates the move that they should make next. There will be an entry for every possible state in the game.